JAVA PROGRAMMING — PART 3

Continuation: exception handling, collections, generics, files, and multithreading

Page 1 — Exception Handling

An exception is an object representing an unusual condition during execution. Handling exceptions helps a program respond meaningfully rather than terminating unexpectedly. Java uses try, catch, finally, throw, and throws.

```
try {
    int number = Integer.parseInt("abc");
    System.out.println(number);
} catch (NumberFormatException e) {
    System.out.println("Enter a valid integer.");
} finally {
    System.out.println("Finished.");
}
```

A try block contains code that may fail; catch handles a matching exception; finally is commonly used for cleanup. Try-with-resources automatically closes resources such as files.

Checked and unchecked exceptions

- **Checked:** compiler requires catching or declaring them, e.g. IOException.
- **Unchecked:** subclasses of RuntimeException, e.g. NullPointerException, are not required to be declared.
- **throw:** throws an exception object; **throws:** declares possible exceptions in a method signature.

Page 2 — Collections Framework

The Collections Framework provides reusable data structures. List is ordered and permits duplicates; Set stores unique elements; Queue supports ordered processing; Map stores key-value associations and is not a subtype of Collection.

```
import java.util.*;

List<String> names = new ArrayList<>();
names.add("Aman");
names.add("Riya");
names.add("Aman");

Set<String> unique = new HashSet<>(names);
Map<String, Integer> scores = new HashMap<>();
scores.put("Aman", 90);
scores.put("Riya", 85);

System.out.println(names.size());  // 3
System.out.println(unique.size()); // 2
System.out.println(scores.get("Aman")); // 90
```

ArrayList is useful for indexed access and dynamic sizing. HashSet is useful for uniqueness. HashMap supports key-based lookup on average. Hash-based collections do not guarantee a meaningful iteration order.

Page 3 — Generics and Sorting

Generics let classes, interfaces, and methods work with specified types while preserving compile-time type safety. List means the list is intended to hold Strings; the compiler can catch an attempt to add an integer.

```
List<Integer> values = new ArrayList<>();
values.add(10);
// values.add("ten"); // compile-time error
```

A generic class can use a type parameter such as T. This reduces casting and catches many type mistakes before execution.

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}
Box<String> box = new Box<>();
box.set("Java");
System.out.println(box.get());
```

Comparable defines natural ordering using compareTo(). Comparator defines an external comparison rule, useful when objects need multiple sorting orders.

Page 4 — File Handling

File handling stores and retrieves information beyond a program's lifetime. Java provides Path and Files in java.nio.file. File operations may throw IOException, so handle or declare it.

```
import java.nio.file.*;

Path path = Path.of("notes.txt");
Files.writeString(path, "Learning Java\n");
String content = Files.readString(path);
System.out.println(content);
```

These convenience methods work well for small text files. Larger files may require buffered or streaming approaches. Relative paths are interpreted from the program's working directory.

Page 5 — Multithreading and Final Revision

A thread is an independent path of execution within a process. Multithreading can help handle concurrent tasks, but shared mutable data needs coordination. Higher-level tools such as ExecutorService are often preferable for managing many tasks.

```
class Task implements Runnable {
    public void run() {
        System.out.println("Task is running");
    }
}
Thread worker = new Thread(new Task());
worker.start();
```

Calling start() asks the JVM to run the task on a new thread; calling run() directly is an ordinary method call on the current thread. Output order across concurrent tasks is not always predictable.

Revision checklist

- Explain try/catch/finally and checked versus unchecked exceptions.
- Choose List, Set, Queue, or Map for a requirement.
- Explain how generics improve type safety.
- Read and write a small text file using Path and Files.
- Explain start() versus run().
- Practise each topic by modifying and running the examples.

What to learn next

After these topics, move to algorithmic problem solving, time complexity, recursion, sorting and searching, linked lists, stacks, queues, trees, graphs, unit testing, and a small Java project. Focus on understanding and modifying code rather than memorizing it.